

ADR 008: Automatic Nix Expression Generation

Status: Active (Q2 2026)

Type: Feature

Created: 2024-07-17

Updated: 2026-04-04

Related-ADRs: 006, 007, 009, 020, 022, 030, 031, 035

Depends-On: ADR 006 Phase 1 (completed)

Revisions

- v0.1: Initial draft (2024-07-17)
- v0.2: Tweak following feedback from NLNet (2024-09-23)
- v0.3: Mark as Phase 3, pending earlier phases (2026-03-23)
- v0.4: Major rewrite. Reframed from “lockfile conversion” to “on-the-fly generation at build time.” Updated with concrete design and implementation plan. (2026-04-04)

Context

What We Have (Phase 1 — Done)

The NixBuilder plugin (ADR 006, implemented Q1 2026) builds applications from hand-crafted `hop3.nix` files. This works well but requires the developer to write a Nix derivation — a significant barrier given Nix’s learning curve [9]. We have 22 production-grade apps with hand-crafted `hop3.nix` files; writing each one took meaningful effort and Nix expertise.

The Problem

The current situation creates a two-tier experience:

- **With `hop3.nix`:** Full reproducibility, hermetic builds, bandwidth-efficient updates (Proposition 1 from the paper). But requires Nix expertise.
- **Without `hop3.nix`:** Fast native builds via the LocalBuilder, but no reproducibility guarantees. This is what most developers will use.

We want to close this gap: give developers the reproducibility benefits of Nix without requiring them to learn the Nix expression language.

Why Not “From Lockfiles”

The earlier framing of this ADR was “generate Nix expressions from lockfiles.” This is misleading for two reasons:

1. **Not all ecosystems have lockfiles.** PHP (`composer.lock`), Go (`go.sum`), and Java (no standard lockfile) would need different approaches.
2. **The output doesn’t need to be a file the developer sees.** The generated Nix expression is an intermediate build artifact, not a configuration file. It should be ephemeral — generated at build time, used to produce the Nix closure, then discarded.

The correct framing is: **the NixBuilder auto-generates a Nix derivation at build time from the project’s existing files, producing the same result as a hand-crafted `hop3.nix` would.**

Decision

When the operator selects `builder = "nix"` in `hop3.toml` but no `hop3.nix` file exists, the NixBuilder will **automatically generate** a Nix derivation at build time by:

1. Detecting the language ecosystem (same detection logic as the LocalBuilder’s toolchain selection)

2. Generating a Nix expression that uses the appropriate Nix ecosystem tool (poetry2nix, dream2nix, node2nix, buildGoModule, etc.)
3. Building the generated expression with nix-build
4. Producing the standard BuildArtifact with RuntimeConfig

The generated Nix expression is **ephemeral** — it lives in a temporary directory during the build and is not committed to the repository. If the developer wants to customize it, they can run `hop3 nix:eject <app>` to materialize the generated expression as a `hop3.nix` file, which they can then modify and commit.

Fallback Hierarchy

`hop3.nix` exists?

- Yes: Use it directly (Phase 1, current behavior)
- No: `builder = "nix"` in `hop3.toml`?
 - Yes: Auto-generate Nix expression from project files
 - No: Use `LocalBuilder` with native toolchains

Design

Per-Language Generators

Each supported language ecosystem gets a generator that produces a Nix expression from project files:

Ecosystem	Input Files	Nix Tool	Maturity
Python	<code>uv.lock</code> , <code>poetry.lock</code> , <code>requirements.txt</code>	<code>poetry2nix</code> / <code>dream2nix</code> / <code>pip2nix</code>	Good (<code>poetry2nix</code> is mature)
Node.js	<code>package-lock.json</code> , <code>yarn.lock</code> , <code>pnpm-lock.yaml</code>	<code>dream2nix</code> / <code>node2nix</code>	Fair (<code>node2nix</code> has known issues, <code>dream2nix</code> improving)
Go	<code>go.mod</code> + <code>go.sum</code>	<code>buildGoModule</code> (built into <code>nixpkgs</code>)	Excellent (native <code>nixpkgs</code> support)
Ruby	<code>Gemfile.lock</code>	<code>bundlerEnv</code> (built into <code>nixpkgs</code>)	Good
Rust	<code>Cargo.lock</code>	<code>crane</code> / <code>naersk</code>	Good
Java	<code>pom.xml</code> , <code>build.gradle</code>	<code>mvn2nix</code> / <code>gradle2nix</code>	Experimental
PHP	<code>composer.lock</code>	<code>composer2nix</code>	Experimental
Static	HTML/CSS/JS files	<code>stdenv.mkDerivation</code> (trivial copy)	Trivial

Generator Interface

```
class NixExpressionGenerator(Protocol):
    """Generates a Nix expression from project files."""

    def detect(self, source_path: Path) -> bool:
        """Can this generator handle this project?"""
        ...

    def generate(self, source_path: Path, config: Hop3Config) -> str:
        """Generate a Nix expression string."""
        ...
```

Each generator produces a complete Nix expression that:

- Takes pkgs as input (from the pinned nixpkgs)
- Declares all dependencies
- Builds the application
- Produces a \$out/hop3/runtime.json with worker commands, env vars, and PATH entries (same contract as hand-crafted hop3.nix)

Example: Generated Python Expression

For a Python project with uv.lock:

```
# Auto-generated by hop3 NixBuilder – do not edit
# Run 'hop3 nix:eject myapp' to materialize and customize
{ pkgs ? import <nixpkgs> {} }:
let
  python = pkgs.python312;
  pythonPackages = python.pkgs;

  # dream2nix or poetry2nix handles dependency resolution
  app = pkgs.callPackage ./dream2nix-generated.nix {};
in
pkgs.stdenv.mkDerivation {
  name = "myapp";
  src = ./.;

  buildInputs = [ app python ];

  installPhase = ''
    mkdir -p $out/hop3
    cp -r . $out/app

    cat > $out/hop3/runtime.json <<EOF
    {
      "workers": { "web": "$out/app/venv/bin/gunicorn app:app --bind 0.0.0.0:\
$PORT" },
      "env_vars": { "PYTHONUNBUFFERED": "1" },
      "path_prepend": ["$out/app/venv/bin"]
    }
    EOF
  '';
```

The nix:eject Command

```
hop3 nix:eject myapp
```

Materializes the auto-generated Nix expression as hop3.nix in the project root, with comments explaining each section. After ejection, the developer owns the file and can customize it. Subsequent builds use the committed hop3.nix (Phase 1 path) instead of auto-generating.

This mirrors the “eject” pattern from Create React App and similar tools.

Is This a Good Idea?

Arguments For

1. **Removes the #1 adoption barrier.** The Nix learning curve is documented as the main obstacle to reproducible build adoption [9]. Auto-generation eliminates this for the common case.

2. **Progressive disclosure.** Developers start with auto-generation (zero Nix knowledge), then eject and customize when they need to (gradual Nix learning).
3. **Leverages existing ecosystem.** poetry2nix, buildGoModule, crane — these tools exist and are maintained by the Nix community. We're composing, not rementing.
4. **Same output contract.** The generated expression produces the same `runtime.json` → BuildArtifact as a hand-crafted one. The rest of the pipeline is unchanged.

Arguments Against / Risks

1. **Generated Nix may not be optimal.** Hand-crafted expressions can use Nix-specific optimizations (custom phases, overlays) that generators can't discover. Mitigation: the `nix:eject` escape hatch.
2. **Ecosystem tool fragility.** node2nix, pip2nix etc. can break on edge cases (native extensions, private registries, monorepos). Mitigation: start with the most mature generators (Python, Go, Rust) and expand cautiously.
3. **Maintenance burden.** Each generator needs to track upstream Nix tool changes. Mitigation: limit the initial set to 3-4 ecosystems.
4. **False sense of reproducibility.** If the generated expression has a bug, the user gets a broken Nix build with cryptic errors and no Nix knowledge to debug it. Mitigation: clear error messages, easy fallback to native builder, `nix:eject` for debugging.

Recommendation

Yes, this is a good idea, with the caveat that initial scope should be limited to the 3-4 most mature ecosystems (Python, Go, Rust, Static). Node.js and Java generators should be marked experimental. The `nix:eject` pattern provides a clean escape hatch that makes the auto-generation low-risk — if it doesn't work for a specific project, the developer ejects and fixes the expression manually.

Implementation Plan

Phase 3a: Python + Go (May 2026)

1. Implement NixExpressionGenerator protocol
2. Python generator using poetry2nix or dream2nix
3. Go generator using buildGoModule
4. `nix:eject` command
5. Test with 5 apps from the evaluation corpus (SearXNG, Miniflux, Gitea, Radicale, Isso)

Phase 3b: Rust + Static + Node.js (June 2026)

1. Rust generator using crane
2. Static generator (trivial mkDerivation)
3. Node.js generator using dream2nix (experimental)
4. Test with additional apps

Phase 3c: Remaining Ecosystems (Q3 2026)

1. Ruby generator using bundlerEnv
2. Java/PHP generators (experimental)
3. Broader testing, documentation

Consequences

Benefits

- Developers get Nix reproducibility without writing Nix
- `builder = "nix"` in `hop3.toml` is all that's needed

- Progressive disclosure via `nix:eject`
- Same BuildArtifact output — no pipeline changes needed

Drawbacks

- Per-ecosystem generator maintenance
- Edge cases in auto-generation may produce confusing errors
- Generated expressions may not be as efficient as hand-crafted ones

Prior Art

- **dream2nix**: Multi-language Nix framework for auto-generating derivations. Active development.
- **nixpacks** (Railway): Generates Nix-based build plans from project detection. Similar concept but for a different deployment model.
- **poetry2nix**: Mature Python-to-Nix tool, widely used in the Nix community.
- **Create React App eject**: The precedent for the “auto-generate, then eject to customize” pattern.
- **Heroku buildpacks**: Auto-detect and build without user configuration — same UX goal, different technology.